CarAI

Genetic Algorithm Optimisation of a Racing Line

Oliwier Zasadni & Mikołaj Kimak

Course project — Biologically Inspired Artificial Intelligence
Silesian University of Technology (Politechnika Śląska)

1. Introduction

This project combines a 2-D top-down car-driving game with an evolutionary AI that learns the optimal racing line. The user draws an arbitrary closed circuit; the genetic algorithm (GA) then finds the trajectory that minimises the estimated lap time while staying within the driveable surface. The result is replayed by an AI-controlled car that shares the same physics engine as the human driver, making it straightforward to compare machine and player performance on the same timer.

The racing line is a well-suited problem for evolutionary computation: the objective is clear (minimise lap time), the constraint is hard (stay on track), and the search space — the lateral position of the line at every point of the circuit — is large, continuous and non-convex. Crucially, every candidate solution is directly interpretable and drawable, so the learning process can be watched in real time.

2. Analysis of the Task

2.1 Approach selection

Three families of solutions were considered:

- Analytical / optimal-control methods (minimum-curvature or minimum-time, solved with QP). Pro: mathematically exact. Con: requires a smooth, differentiable model and a convex formulation — neither holds for an arbitrary hand-drawn track with a penalty-based constraint.
- Reinforcement learning. Pro: learns to control the car end-to-end. Con: sample-inefficient, unstable to tune, and the result is a black-box policy rather than an inspectable line.
- Genetic algorithm over the racing line. Pro: gradient-free, robust to non-differentiable fitness, and the genome is the racing line — every individual is directly drawable. Con: needs many fitness evaluations.

The GA approach was chosen. It matches the educational aim (each operator is observable), tolerates the penalty-based fitness function, and produces a human-readable result that can be replayed immediately in the game.

2.2 Problem encoding

The track centreline is divided into N = 50 evenly-spaced cross-sections. A chromosome is a vector of 50 real numbers d_i in [0, 1]; gene d_i is the normalised lateral offset at cross-section i (0 = inner edge, 1 = outer edge). The 50 control points are interpolated by a periodic cubic spline into a smooth closed trajectory of 150 points, and a physics model evaluates the lap time. The GA minimises that estimate.

3. AI System Specification

3.1 Chromosome and population

Each individual is a 1-D NumPy array of 50 floats. The population is a (POP_SIZE × 50) matrix. Default: POP_SIZE = 100, 200 generations.

3.2 Fitness function

The fitness of a chromosome is its estimated lap time in seconds plus a large penalty for any trajectory point outside the corridor. Lower is better. The pipeline:

- Trajectory reconstruction. The 50 gene values are interpolated with a periodic cubic spline (SciPy CubicSpline, bc_type="periodic") into 150 waypoints.
- Curvature. Menger formula at each point: $\kappa = 4 \cdot \text{Area}(p_1 p_2 p_3) / (|p_1 p_2| \cdot |p_2 p_3| \cdot |p_1 p_3|)$, $R = 1/\kappa$.
- Corner-speed limit. Friction circle: $v_{max} = \sqrt{(\mu \cdot g \cdot R)}$ with $\mu = 0.7$, $g = 9.81$ m/s^2.
- Feasible speed profile. Forward and backward passes enforce $v_j \le \sqrt{(v_i^2 + 2 \cdot a \cdot ds)}$ with MAX_ACCEL = 8 and MAX_DECEL = 15 m/s^2; repeated 3 times for closed-loop convergence; floored at MIN_SPEED = 2 m/s.
- Lap time. $T = \Sigma_i \, ds_i / v_{avg,i}$.
- Boundary penalty. 500 s per metre outside the corridor (gradient term) + 1000 s per off-track point (flat term). Any excursion dominates the score and is selected out within a few generations.

3.3 Genetic operators

- Selection. Tournament selection, size k = 5. Two parents are drawn independently for every offspring, giving a mild selection pressure that preserves diversity.
- Crossover — three variants tested: (a) Single-point: one random cut, swap tails. (b) Multi-point: 3 random cuts, alternate segments. (c) Uniform: each gene swapped independently with p = 0.5.
- Mutation. Gaussian perturbation: d_i += N(0, σ) with σ = 0.05, applied per gene with probability MUTATION_RATE (default 0.10); result clipped to [0, 1].
- Elitism. Top 2 individuals copied unchanged into the next generation.

3.4 Godot → Python interface

When the AI is launched, Godot exports the current circuit to track_data.json (centreline, corridor half-width, inner/outer boundary polygons, pixel-to-metre scale). The GA writes its current best line to current_best.json every generation (atomic tmp→rename to prevent partial reads) and Godot polls this file every 0.5 s to draw the evolving racing line in real time. On completion, best_line.json carries the full per-waypoint command list (position, target speed, heading, throttle, brake) that the in-game AI car replays.

3.5 AI driving — replaying the evolved line

The Godot AI car follows the evolved waypoints through the same physics engine as the human driver: a bicycle-model with lateral-grip / slip, speed-sensitive steering and genuine tyre-slip consequences. The AI writes only to the throttle and steering inputs consumed by _physics_process — never position or velocity directly — so the car obeys the same engine, grip and collision model the player experiences. A look-ahead heading controller sets the steering target; a speed-profile follower controls throttle and brake.

4. Experiments

4.1 Methodology

Three experiment sets study how the GA's main control parameters affect solution quality (final estimated lap time) and convergence speed. All other settings are held at their defaults and every configuration is evaluated on the same circuit, so observed differences are attributable to the varied parameter alone. Each configuration is run with 3 independent seeds [42, 123, 7]; shaded bands in line graphs show the seed spread; error bars in bar charts show ±1 standard deviation.

- Set A — Crossover operator: single-point vs multi-point (3 cuts) vs uniform.
- Set B — Mutation rate: p in {0.01, 0.05, 0.10, 0.20, 0.40}.
- Set C — Population size: {20, 50, 100, 200}.

Derived metrics. Population diversity = mean per-gene standard deviation across the population (falls as the population converges to similar solutions). Convergence generation = first generation whose running-best is within 2% of the final running-best. Improvement per generation = drop in running-best from one generation to the next.

4.2 Results and analysis

Overall GA behaviour — default configuration

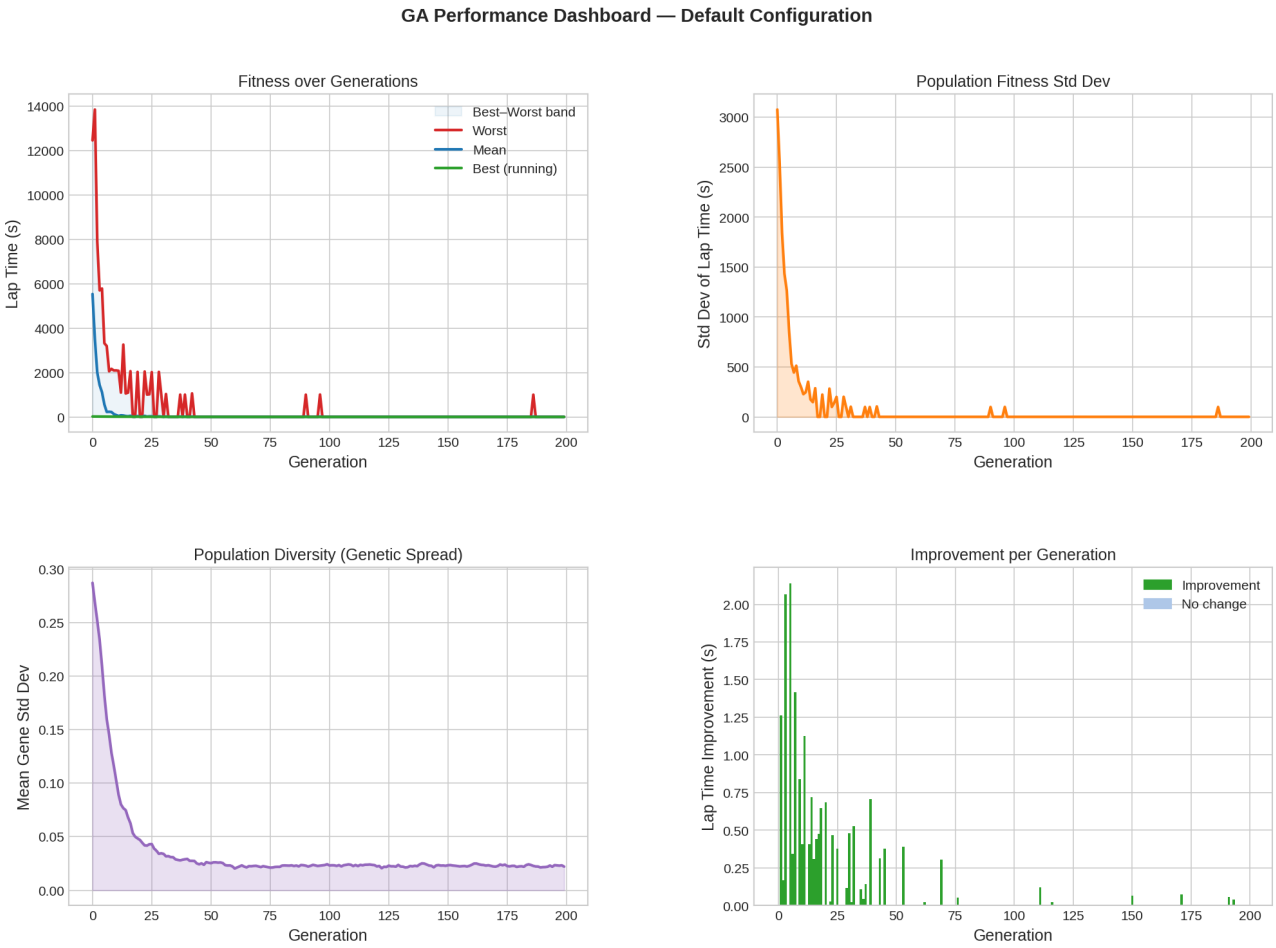

Figure 1. GA performance dashboard (population 100, mutation 0.10, uniform crossover, 200 generations). Top-left: best / mean / worst fitness over generations (worst starts near 14 000 s due to off-track penalties). Top-right: fitness standard deviation. Bottom-left: population genetic diversity. Bottom-right: improvement per generation.

The dashboard reveals the algorithm's characteristic behaviour. The worst and mean fitness start in the thousands of seconds because most random initial lines leave the corridor and incur the 1000 s-per-point

penalty; selection eliminates these almost immediately and by roughly generation 40 the entire population is on-track. The improvement-per-generation panel shows that essentially all useful gain happens in the first ~50 generations; after that only sporadic small refinements occur. The diversity panel mirrors this: genetic spread drops sharply from ~0.29 to a small residual (~0.02) and then plateaus — the population has converged but never fully collapses, thanks to mutation and tournament selection. Occasional red spikes in the worst-fitness trace are mutated individuals briefly leaving the track, incurring the penalty and being immediately bred out.

Set A — Crossover operator

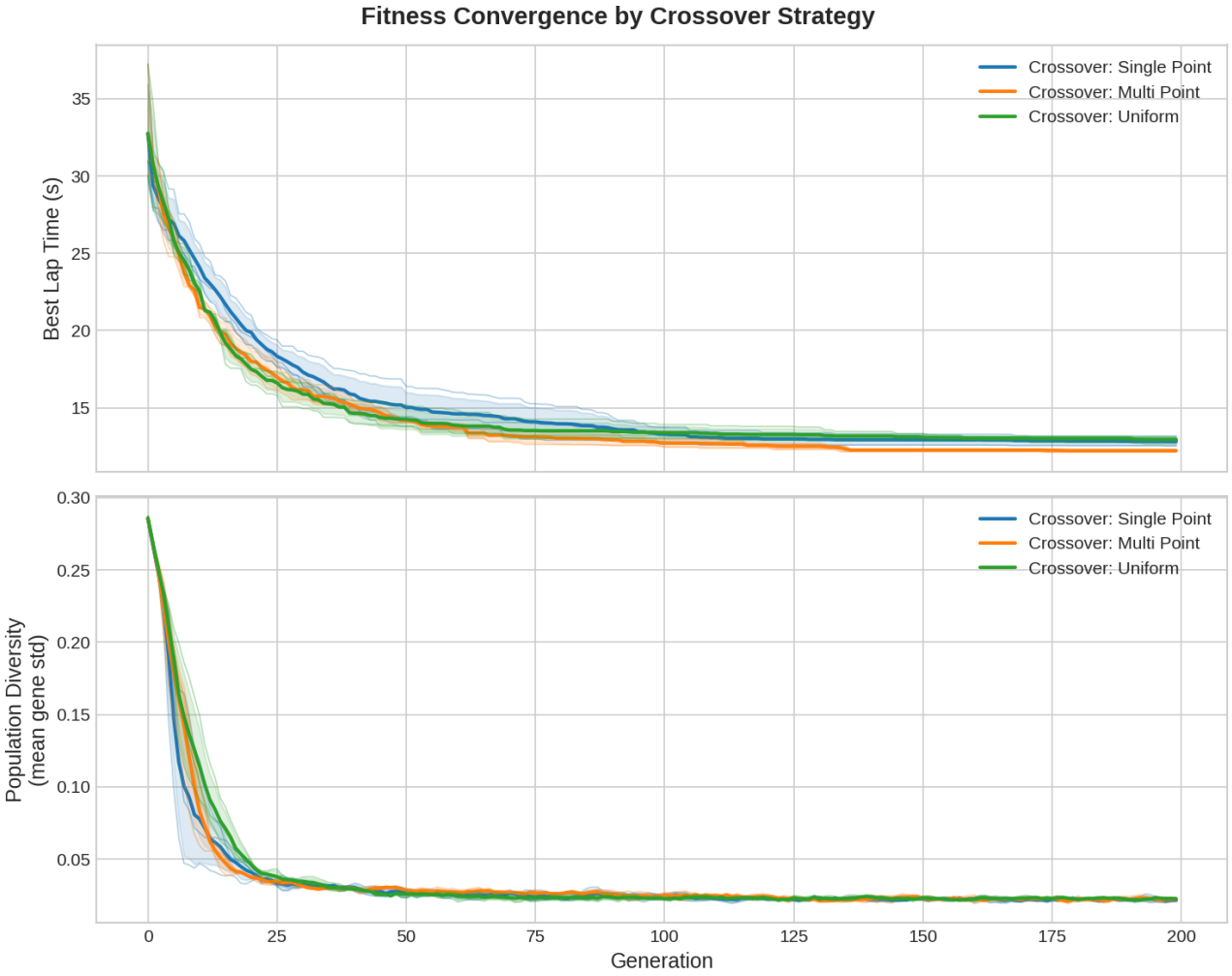

Figure 2. Fitness convergence (top) and population diversity (bottom) by crossover operator, averaged over 3 seeds; shaded bands show the seed spread.

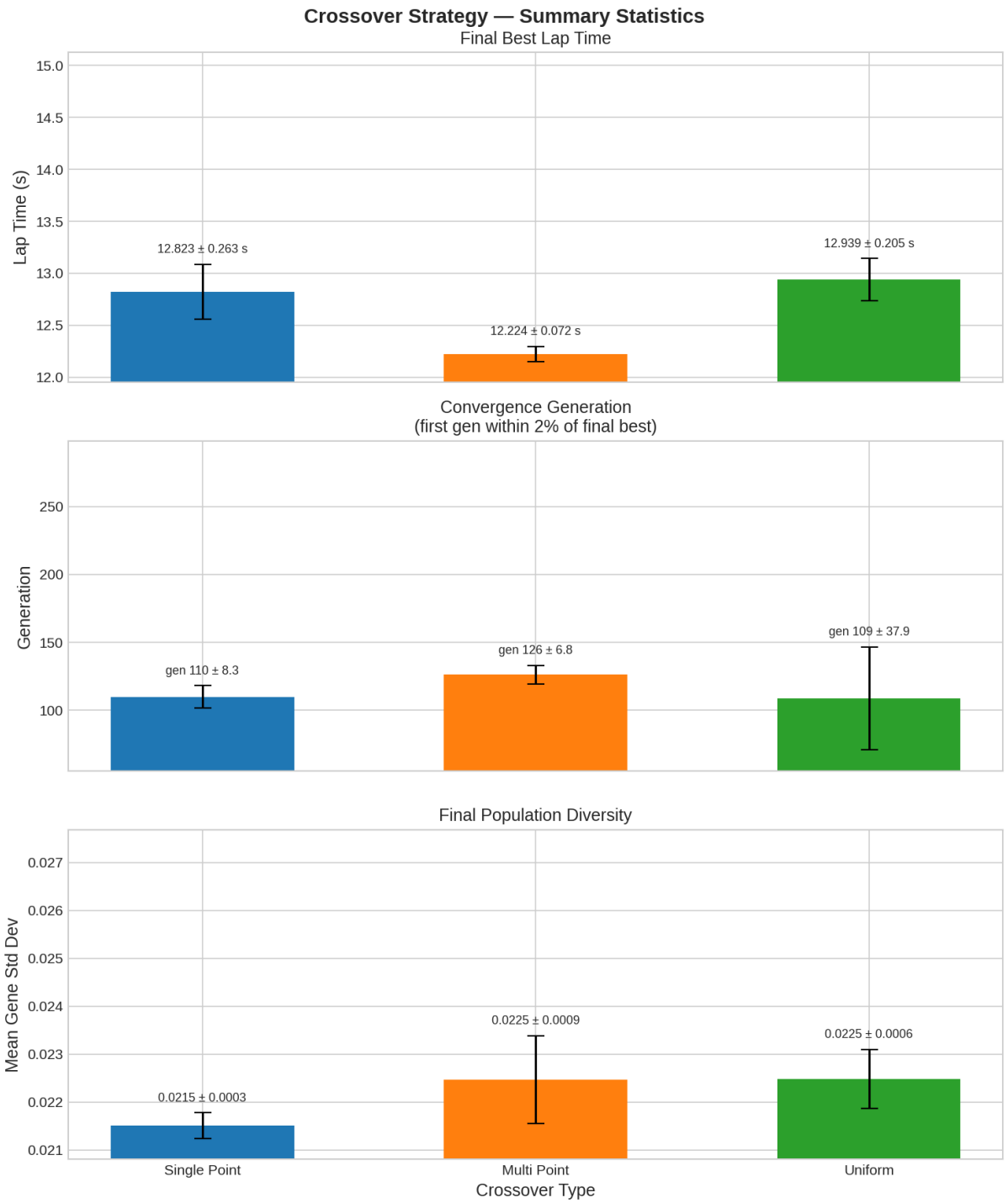

Figure 3. Crossover summary: final best lap time, convergence generation, and final population diversity (mean ± std over 3 seeds).

All three operators converge to a similar quality range (Figure 2), as expected with elitism and 200 generations. The differences lie in reliability and final value (Figure 3). Multi-point crossover produced the best and most consistent result — 12.224 ± 0.072 s — ahead of single-point (12.823 ± 0.263 s) and uniform (12.939 ± 0.205 s). Multi-point recombines contiguous track segments, which preserves locally-good cornering arcs while still mixing material from both parents; this spatial locality is well matched to the genome structure. Uniform crossover maintained higher diversity early (slowest-falling green curve in Figure 2, lower panel) but that extra exploration did not translate into a better final lap time. Conclusion: multi-point crossover is the best default for this spatially-structured genome.

Set B — Mutation rate

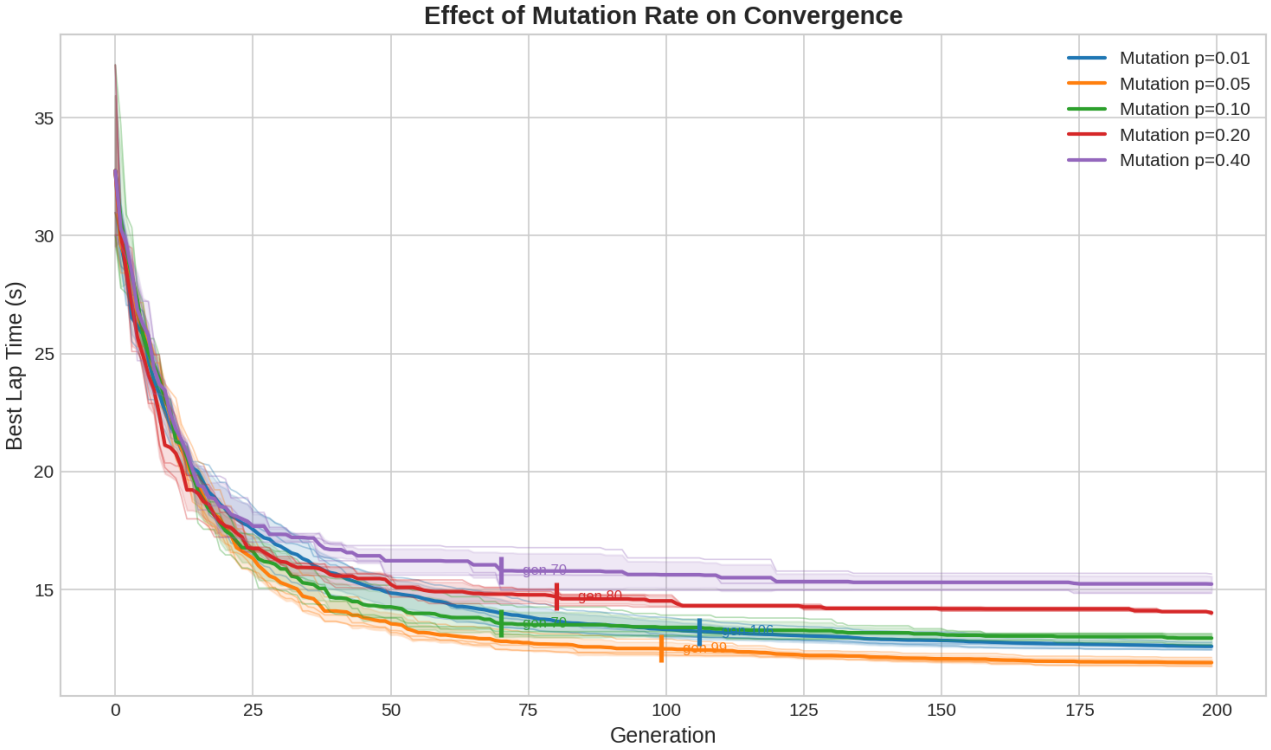

Figure 4. Effect of mutation rate on convergence (best lap time vs generation, 3-seed average). Vertical ticks mark each rate's convergence generation.

Mutation rate shows a clear optimum (Figure 4). p = 0.05 produced the best final lap time (~12.2 s), with p = 0.01 and p = 0.10 close behind (~12.7–13.0 s). Higher rates degrade the result markedly: p = 0.20 settles around 14 s and p = 0.40 around 15.2 s. Excessive mutation continually disrupts good solutions and prevents the population from settling — it acts almost like a random walk around a slowly-shifting mean rather than a directed search. The convergence markers illustrate a complementary effect: high rates appear to "converge" early (gen ~70–80) only because they stabilise on a worse plateau they cannot escape, whereas p = 0.05 keeps improving until ~gen 99 and reaches the lowest value. Conclusion: a small rate (~0.05) best balances refinement against disruption; the default of 0.10 is reasonable but slightly conservative.

Set C — Population size

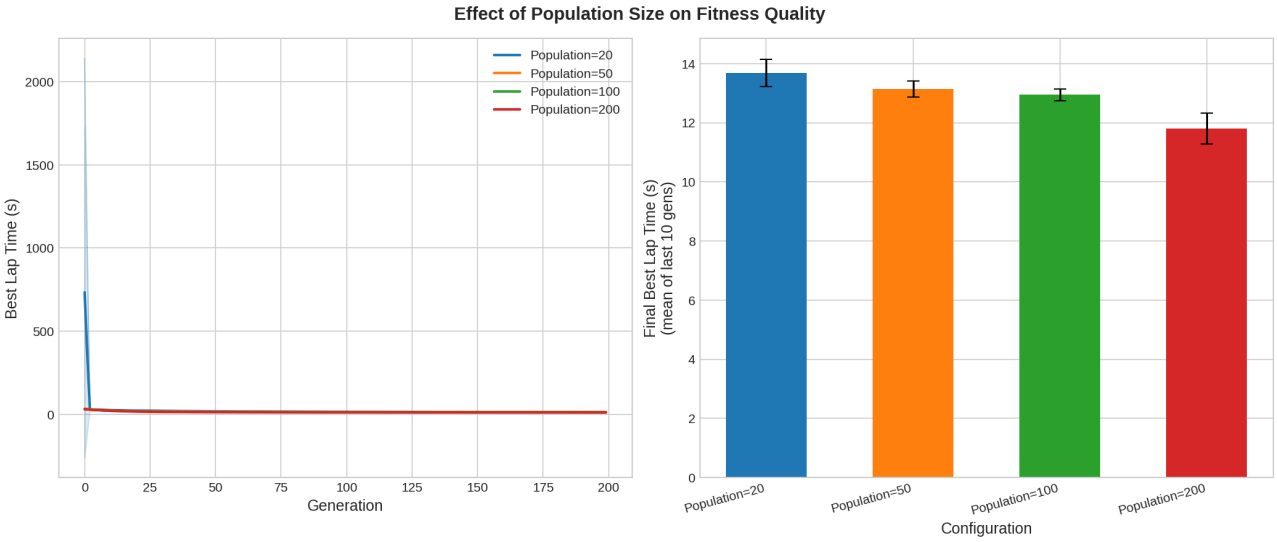

Figure 5. Effect of population size. Left: best lap time vs generation. Right: final best lap time (mean of last 10 generations) with seed error bars.

Final lap time falls monotonically with population size (Figure 5): ~13.7 s at pop-20, ~13.1 s at pop-50, ~13.0 s at pop-100, and ~11.8 s at pop-200. A larger population samples the search space more densely and preserves more useful genetic material, reducing the risk of premature convergence to a mediocre local optimum. The benefit has diminishing returns relative to cost: the 20→200 step is a 10× increase in evaluations per generation for roughly a 14% lap-time gain, and the 50→100 step barely moves the needle. Conclusion: population 100 is the best quality/cost trade-off; 200 is worth using when the best possible line is required and the additional compute is acceptable.

Combined reading. The three sets are consistent with each other and with GA theory: almost all improvement occurs in the first ~50 generations (Figure 1); the crossover operator matters mainly for solution consistency rather than reachability; and the two parameters most affecting converged quality are a moderate-to-low mutation rate and a sufficiently large population. A strong default suggested by these experiments is multi-point crossover, mutation rate ~0.05, population 100–200.

5. Summary, Conclusions and Future Work

5.1 What was achieved

A complete end-to-end pipeline is operational: the user draws a circuit, the GA evolves a racing line against a physics-based fitness function with a hard on-track constraint, and the result is replayed by an AI car timed by the same lap system as the human driver. Key problems resolved during development:

- Unified AI / manual control path. The AI was reworked to write only to the throttle and steering inputs consumed by the car's normal physics loop, so it obeys the same engine, grip and collision model as the player. A look-ahead heading controller eliminates angle-wrapping bugs.
- Mid-lap freeze fixed. A waypoint-index exhaustion bug was resolved by wrapping the index modulo the waypoint count, closing the trajectory loop on the Python side, and flooring all target speeds at MIN_SPEED = 2 m/s.
- Racing line constrained to track surface. The GA was given the true corridor boundaries; cross-section sampling clips genes to the driveable surface; a strong boundary penalty (gradient + flat) eliminates any residual off-track solutions within a few generations.
- Full experiment and visualisation harness. A reproducible runner executes Sets A/B/C across 3 seeds each and renders five publication-quality figures providing the quantitative basis for §4.

5.2 Conclusions

The genetic-algorithm approach solves the racing-line optimisation problem effectively and remains highly legible throughout: every generation produces a drawable, driveable candidate line. The parameter study yields clear, theory-consistent guidance. The physics-based fitness function with a dominant boundary penalty proved an effective way to encode a hard constraint into an unconstrained optimiser, eliminating off-track solutions within ~40 generations regardless of configuration.